

06/10/26

Day - 8

ingress rule $\rightarrow$ inbound rule.

egress rule $\rightarrow$ outbound rule.

EC2 $\rightarrow$ separate security group.

configure panos

VC - security-group-id = [aws-security-group, EC2-SG]

0.0.0.0/0 => allow all

Terraform Variables:

Purpose => Reusability

variables.tf => create new file

variable "ami-reference" {

type = string

default = "ami-12345678"

}

least priority

default value assigned

we can create multiple variables file.

Environment variable
directly pass variable
terraform.tf variable

variable define panda file => terraform.tfvars

you can only store variable

.tfvars => ^{variable} initial .tf file in which reference use panda
value kulluppan.
most Priority

variable .tf create agithu apparam overruddu agom

tfvars -la

default value only you want => only ^{variable} .tf file

Instance type change again pathu $\Rightarrow$ instance
destroy again.

AMI change again pathu $\Rightarrow$ instance destroy - again.

How to search any argument value:

output.tf $\Rightarrow$ create new file.
snippet

```
output "ec2-public-ip" {  
  value = aws-ec2-instance-test.ec2-public-ip  
}
```

Connect EC2 instance without directly.

ssh -i "private key file path" username@public ip

ubuntu $\Rightarrow$ default username $\Rightarrow$ ubuntu

windows $\Rightarrow$ default username $\Rightarrow$ administrator

Kubernetes

who is first developing
the kubernetes? Google

(orchestration tool)

kubernetes is an open source container management tool which automates container deployment, container (de) scaling & container load balancing. It orchestrates the docker.

→ organize.

features:

1. Horizontal scaling
2. Self healing
3. Automatic rollout & rollback

docker ⇒ containerization tool.

kubernetes ⇒ docker top la mukkam. So, kubernetes
docker-illa work sollu athu pannum.

Kubernetes ⇒ untl kubernetes engine down.

launch the containers again and again.
the containers crash athu apdina launch
pannile mukkam.

Kubernetes ⇒ only one requirement illuku vendatha polum.

Kubernetes ⇒ multiple computer-la ^{inudhu} container launch pandathuma
kubernetes one computer-la ^{inudhu} remaining-a
access pannum.

master node ⇒ AWS responsibility.

worker node ⇒ customer responsibility.

Cluster

1) Master node (or) control plane

Components:

1. API server $\Rightarrow$ frontend
[Kube API] request passing

Commands $\Rightarrow$ kubectl

2. Scheduler $\Rightarrow$ job yaan jarukunu assign pannuvanu
 $\rightarrow$ proper a jarukunu check pannum.

3. Controller - manager $\Rightarrow$ desired = Actual

- Types
- 1. node controller $\Rightarrow$ check health $\rightarrow$ check fault and recreate
 - 2. deployment controller $\rightarrow$ check deployment strategy
 - 3. replica set controller $\rightarrow$ pod running a jarukunu check pannum.
 - 4. Job controller $\rightarrow$ given job properly kulla or not check pannum.

4. Etcd $\Rightarrow$ Key-Value database

$\rightarrow$ The data stored about which image which pod running.

2) Worker Node:

Components

1. Pod $\Rightarrow$ one or more containers:

eg: 2 containers $\Rightarrow$ same ip
same volume
same n/w.
(ie) one n/w
one ip
one volume

allocate ~~memory~~ and ip,
volume to pod

shared n/w shared volume.

2. Container runtime $\Rightarrow$ Containerd $\Rightarrow$ current version of Kubernetes
previous version $\Rightarrow$ Docker

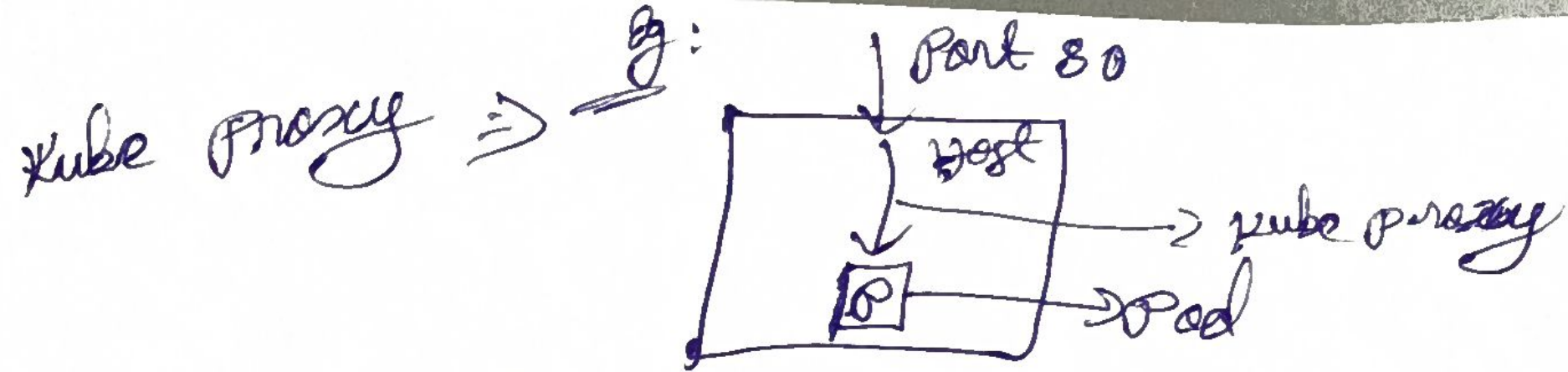
Some Docker commands also working

3. Kubelet $\Rightarrow$ worker node kanna nadanthalum master node kitta sollum. Establish the connection b/w master & worker

4. Kube Proxy $\Rightarrow$ used for expose the service port

pod $\Rightarrow$ Connection of Containers

pod can launch single (or) multiple containers



which port - xu pod - pod redirect pod

Problems with scaling up the Containers

Containers could not communicate with each other.

Containers had to be deployed appropriately.

Containers had to be managed carefully.

Auto Scaling was not possible.

Distributing traffic was still challenging.

Docker Swarm $\Rightarrow$ Docker orchestration tool.